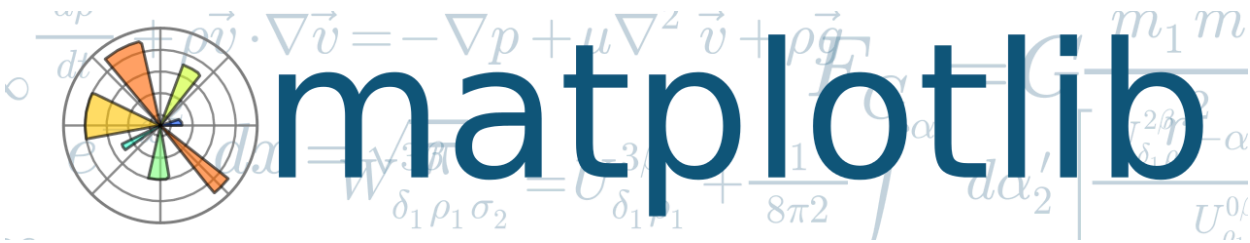


Introduction to Matplotlib :-

Matplotlib is a popular open-source Python library used for creating data visualizations. It helps transform numerical data into graphs and charts, making it easier to understand patterns, trends, and relationships.

Matplotlib is built on NumPy and is widely used in Data Science, Machine Learning, Artificial Intelligence, Scientific Computing, and Business Analytics.

History of Matplotlib :-

Matplotlib was created by **John D. Hunter** in **2003**. He developed it to provide Python with a powerful and easy-to-use plotting library for scientific and engineering applications.

Over the years, Matplotlib has become one of the most widely used visualization libraries in Python. It is the foundation for many other visualization libraries, including Seaborn.

📅 Timeline

- **2003** – Matplotlib was created by John D. Hunter.
- **2005** – Became popular among scientists and researchers.
- **2010+** – Widely adopted in Data Science and Machine Learning.
- **Present** – One of the most popular Python libraries for data visualization.

Why Matplotlib was Developed :-

Before Matplotlib, Python did not have a simple and powerful library for creating high-quality graphs and charts. Developers and researchers needed an easy way to visualize numerical data for analysis and presentations.

Matplotlib was developed to solve this problem by providing a flexible and user-friendly plotting library.

Why Matplotlib :-

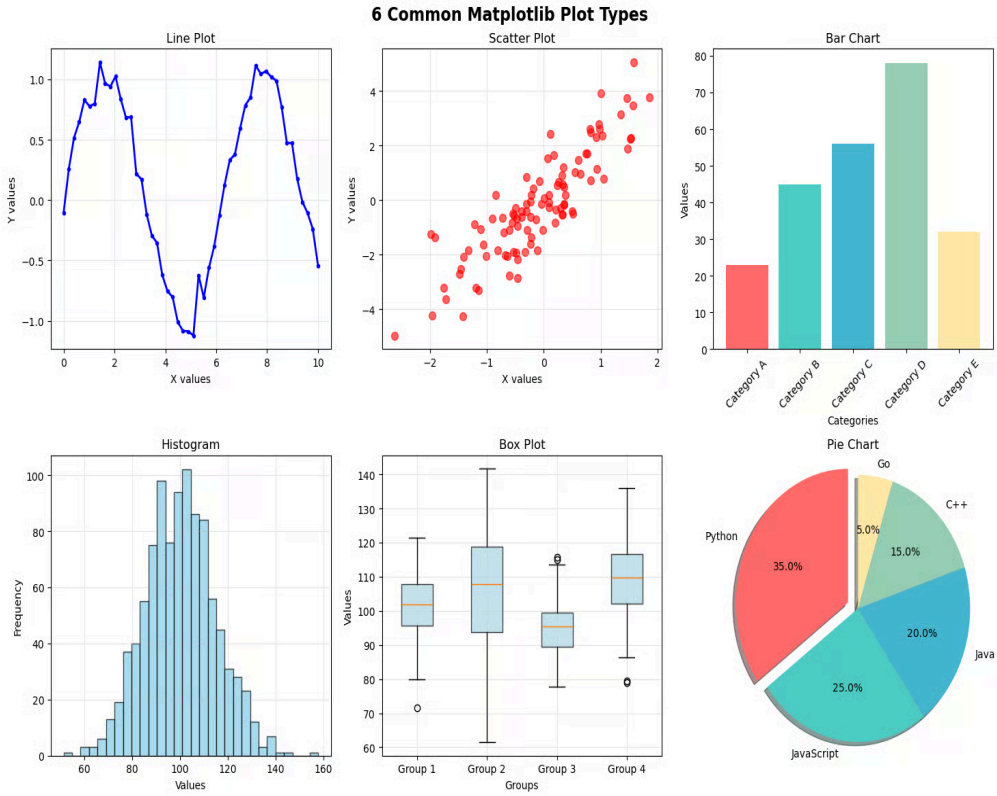
- Convert numerical data into visual charts.
- Make data easier to understand and analyze.
- Create publication-quality graphs.
- Support scientific, engineering, and business applications.
- Integrate seamlessly with NumPy and Pandas.

Features of Matplotlib :-

Matplotlib provides a wide range of features for creating and customizing visualizations. It is one of the most flexible plotting libraries in Python.

Key Features :-

- Supports various chart types (Line, Bar, Pie, Scatter, Histogram, etc.).
- Easy to customize colors, labels, titles, and legends.
- Integrates seamlessly with NumPy and Pandas.
- Supports subplots for displaying multiple charts in one figure.
- Allows saving plots in formats like PNG, JPG, PDF, and SVG.
- Produces high-quality graphs suitable for reports and presentations.
- Free, open-source, and cross-platform.



Applications of Matplotlib :-

Matplotlib is widely used to visualize data in various fields.

- 📊 Data Analysis and Data Science
- 🤖 Machine Learning
- 🧠 Artificial Intelligence
- 📈 Business Analytics
- 💹 Financial Analysis and Stock Market
- 🔬 Scientific Research
- 🏥 Healthcare and Medical Data Analysis
- 📚 Education and Research
- ⚙️ Engineering and Simulations
- 📉 Statistical Data Visualization

Installing and Importing Matplotlib :-

Before using Matplotlib, you need to install it. If you are using Google Colab, Matplotlib is already installed.

Install Matplotlib -

pip install matplotlib

Import Matplotlib -

The `pyplot` module is commonly used for creating charts and graphs. It is usually imported with the alias `plt`.

import matplotlib.pyplot as plt

```
import matplotlib.pyplot as plt
```

```
import matplotlib

print("Matplotlib Version:", matplotlib.__version__)
```

```
Matplotlib Version: 3.10.0
```

1. Line Plot :-

A line plot is used to show trends or changes in data over time. It is one of the most commonly used charts in Matplotlib.

Syntax :-

plt.plot(x, y)

The `plot()` function provides several customization options such as color, marker, line style, and line width.

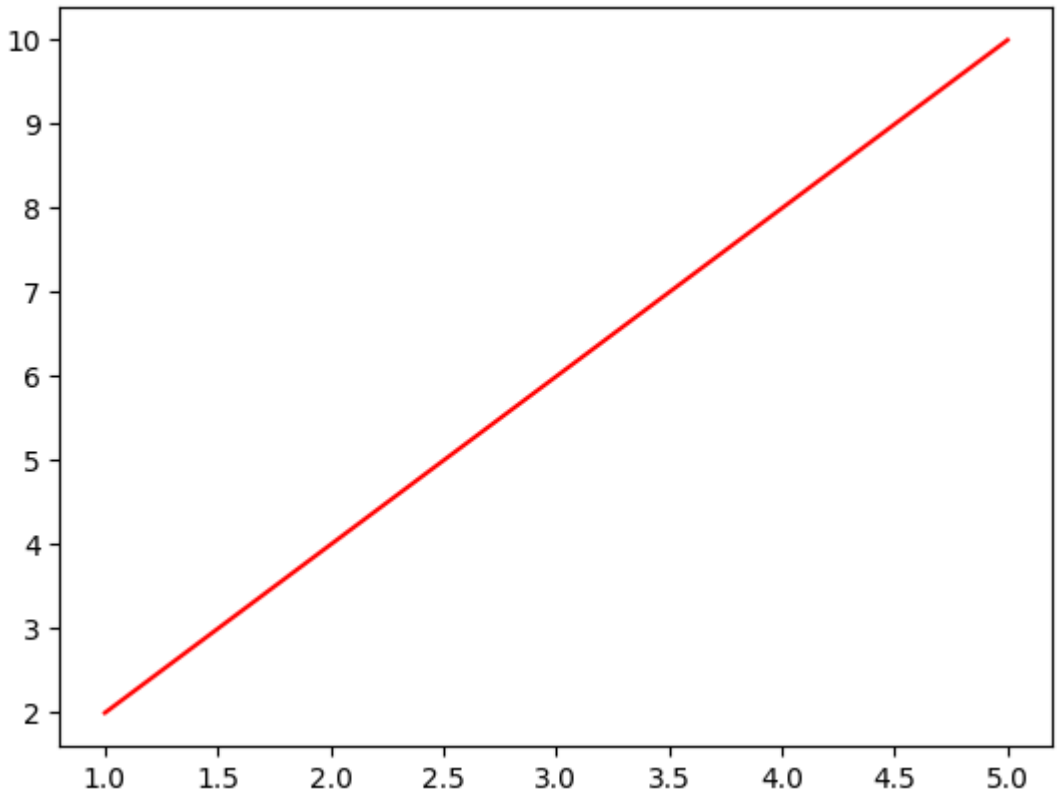
Common Parameters :-

- color – Changes the line color.
- marker – Adds markers to data points.
- linestyle – Changes the style of the line.
- linewidth – Changes the thickness of the line.

Change Line Color :-

Description: Draw a line plot with a red line.

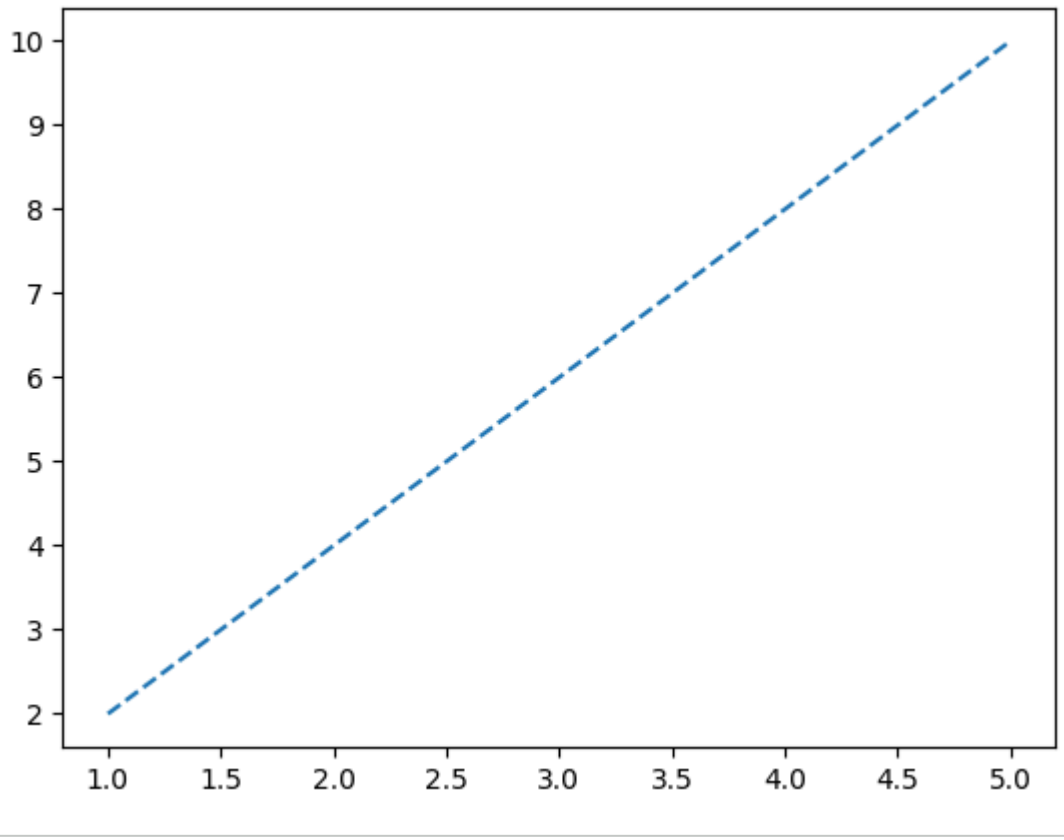
```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

plt.plot(x, y, color="red")
plt.show()
```



Change Line Style :-

Description: Draw a dashed line.

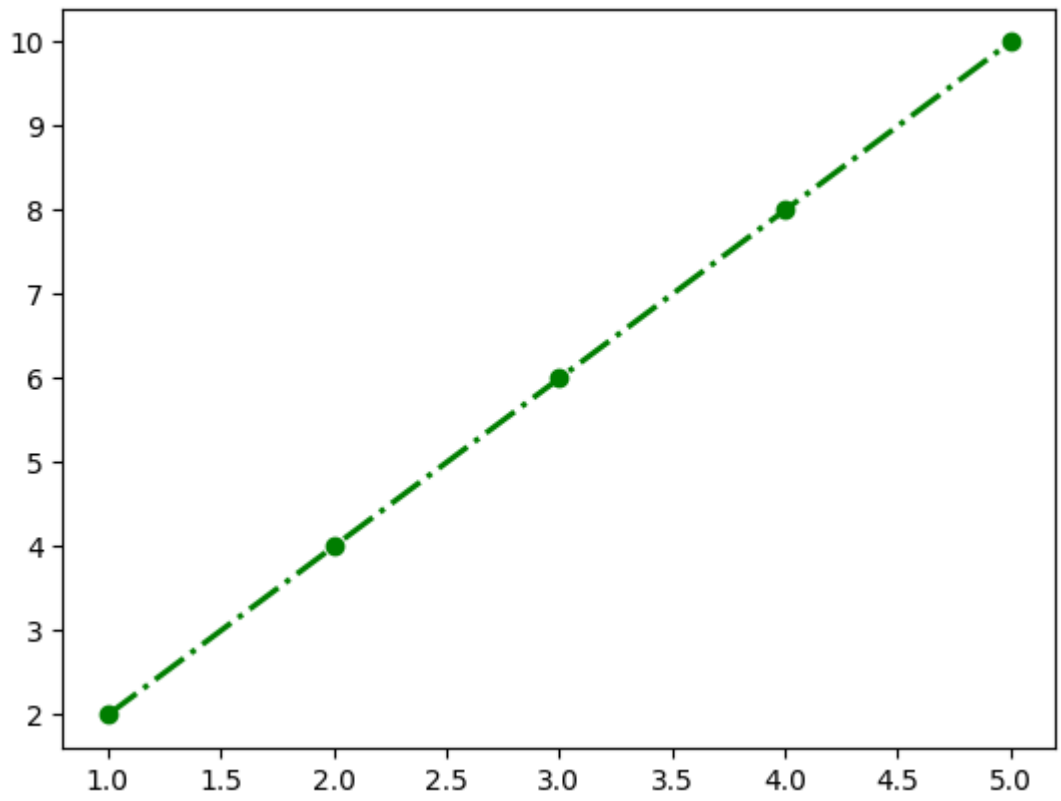
```
plt.plot(x, y, linestyle="--")
plt.show()
```



Combine Customizations :-

Description: Customize the line using multiple parameters.

```
plt.plot(x, y,
         color="green",
         marker="o",
         linestyle="-.",
         linewidth=2)

plt.show()
```



Labels and Title :-

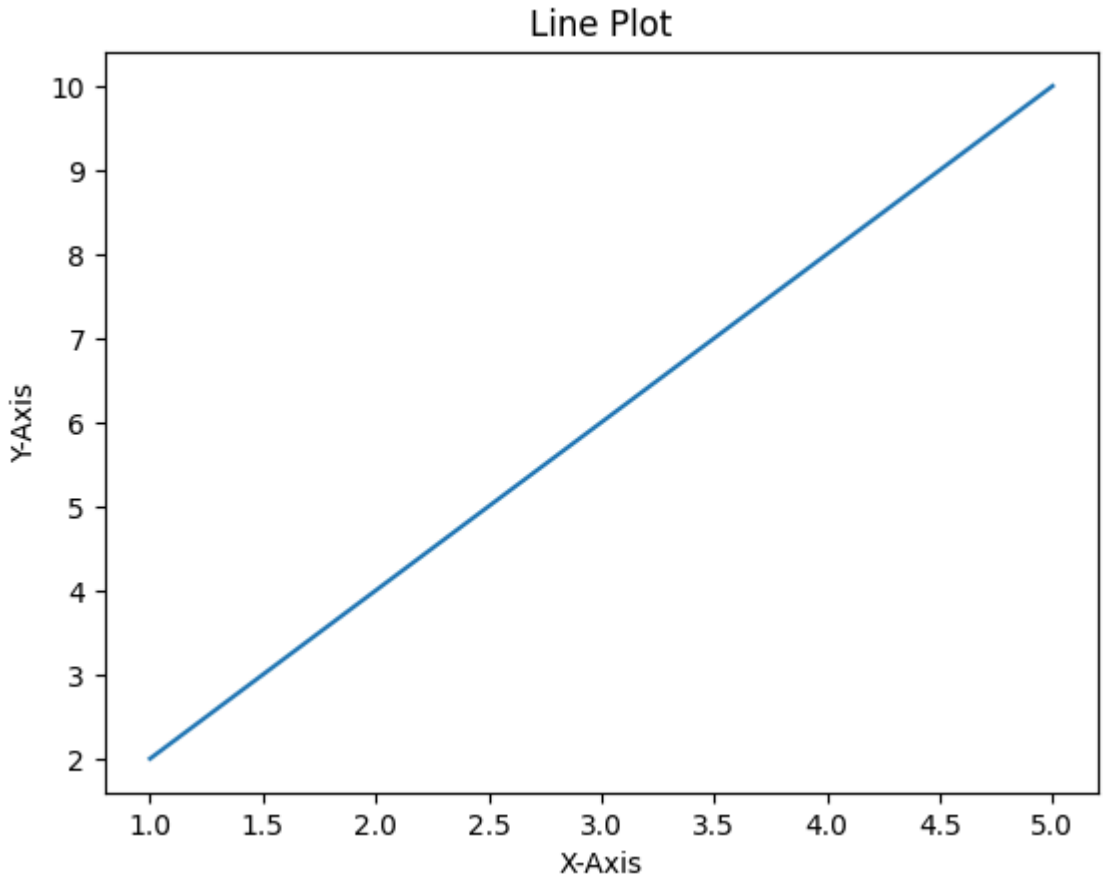
Labels and titles make a graph easier to understand by describing the data being displayed.

Matplotlib provides the following functions:

- `xlabel()` – Sets the label for the x-axis.
- `ylabel()` – Sets the label for the y-axis.
- `title()` – Sets the title of the graph.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
```

```
plt.plot(x, y)
plt.xlabel("X-Axis")
plt.ylabel("Y-Axis")
plt.title("Line Plot")
plt.show()
```



Grid :-

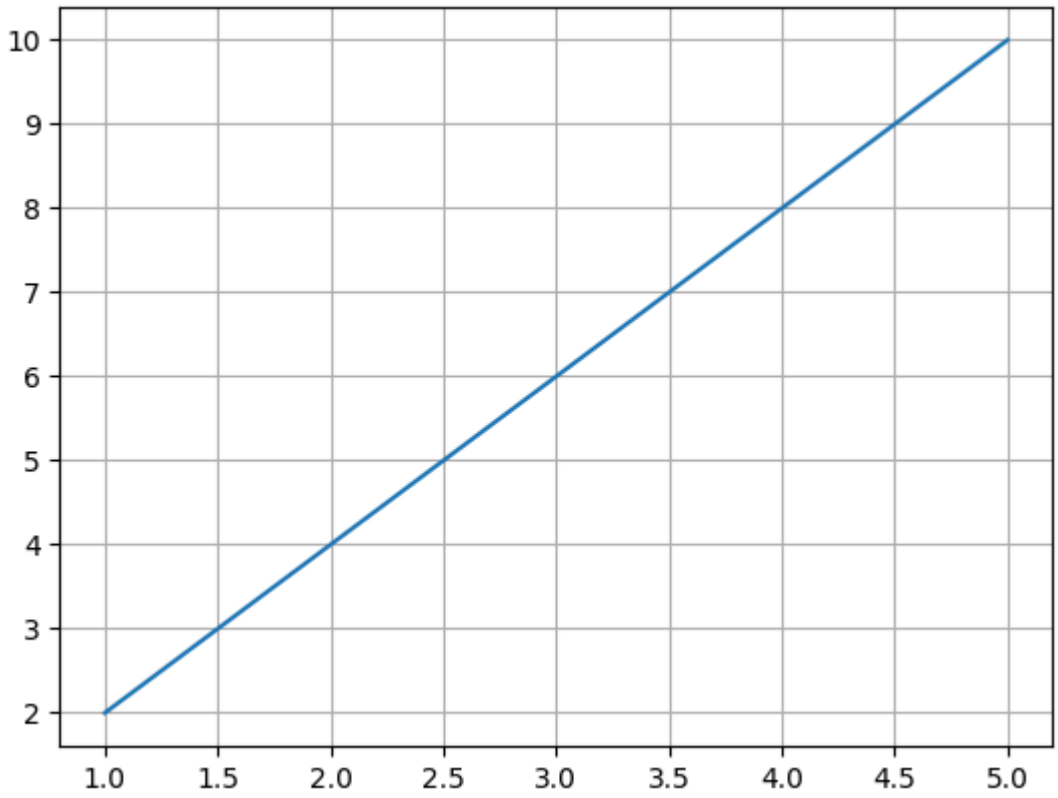
A grid makes a graph easier to read by adding horizontal and vertical reference lines.

The grid() function is used to display a grid on the plot.

Syntax -

plt.grid()

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

plt.plot(x, y)
plt.grid()

plt.show()
```



2. Bar Chart :-

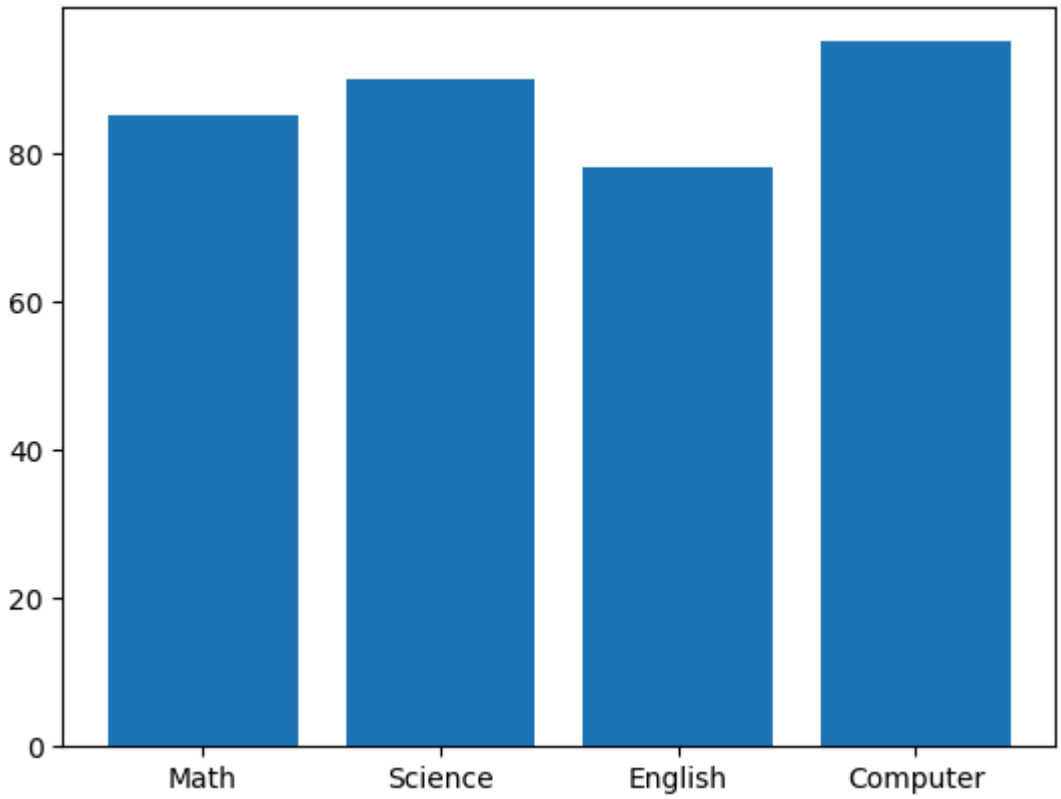
A bar chart is used to compare values across different categories. Each bar represents a category, and its height represents the corresponding value.

Syntax :-

plt.bar(x, height)

The `bar()` function is used to create a vertical bar chart.

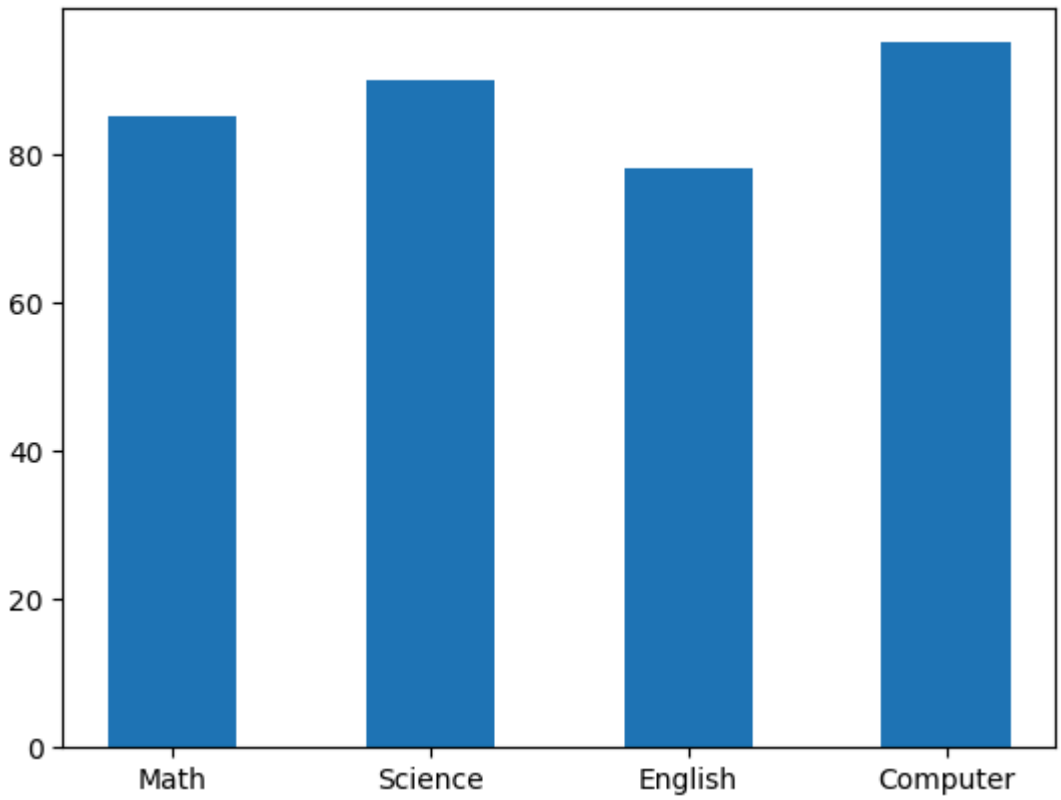
```
import matplotlib.pyplot as plt

subjects = ["Math", "Science", "English", "Computer"]
marks = [85, 90, 78, 95]

plt.bar(subjects, marks)

plt.show()
```



Change Bar Width :-

Description: Adjust the width of the bars.

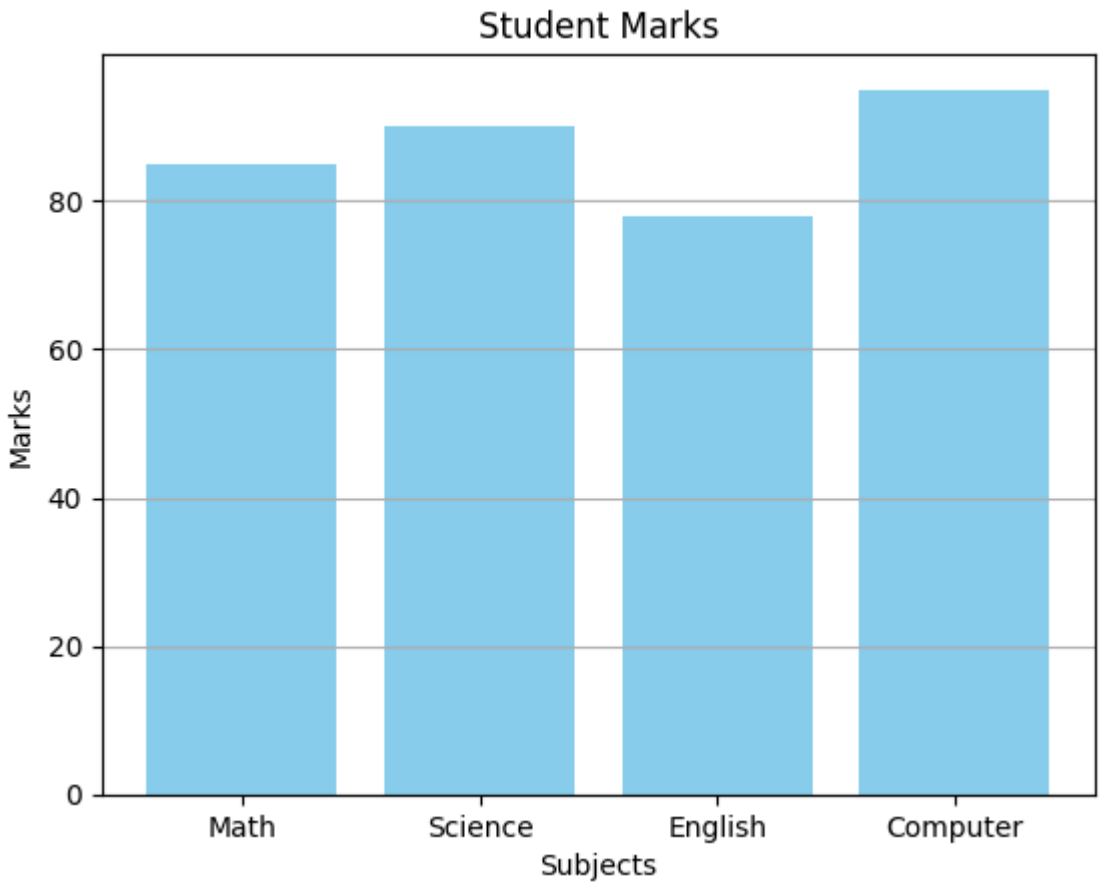
```
plt.bar(subjects, marks, width=0.5)

plt.show()
```



Complete Bar Chart :-

Description: Add a title, axis labels, and a grid.

```
plt.bar(subjects, marks, color="skyblue")

plt.title("Student Marks")
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.grid(axis="y")

plt.show()
```



3.Pie Chart :-

A pie chart is used to show how different categories contribute to a whole. Each slice represents a category, and its size is proportional to its value.

Syntax :-

plt.pie(x)

The `pie()` function is used to create a pie chart.

Basic Pie Chart :-

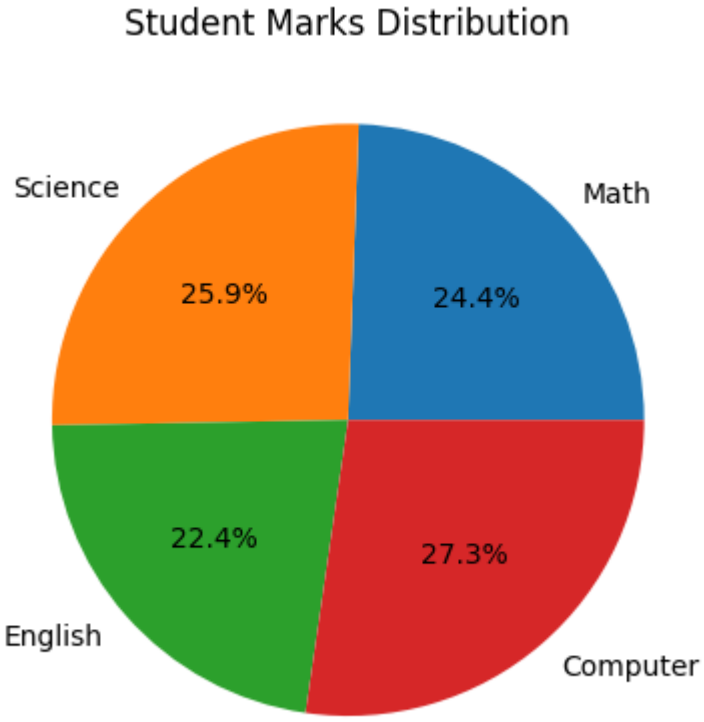
Description: Create a simple pie chart.

```
import matplotlib.pyplot as plt

subjects = ["Math", "Science", "English", "Computer"]
marks = [85, 90, 78, 95]

plt.pie(marks)

plt.show()
```

Display Percentages :-

Description: Show the percentage of each slice.

```
plt.pie(marks, labels=subjects, autopct="%1.1f%%")
plt.title("Student Marks Distribution")

plt.show()
```



4.Histogram :-

A histogram is used to show the distribution of numerical data by grouping values into intervals called bins.

Syntax :-

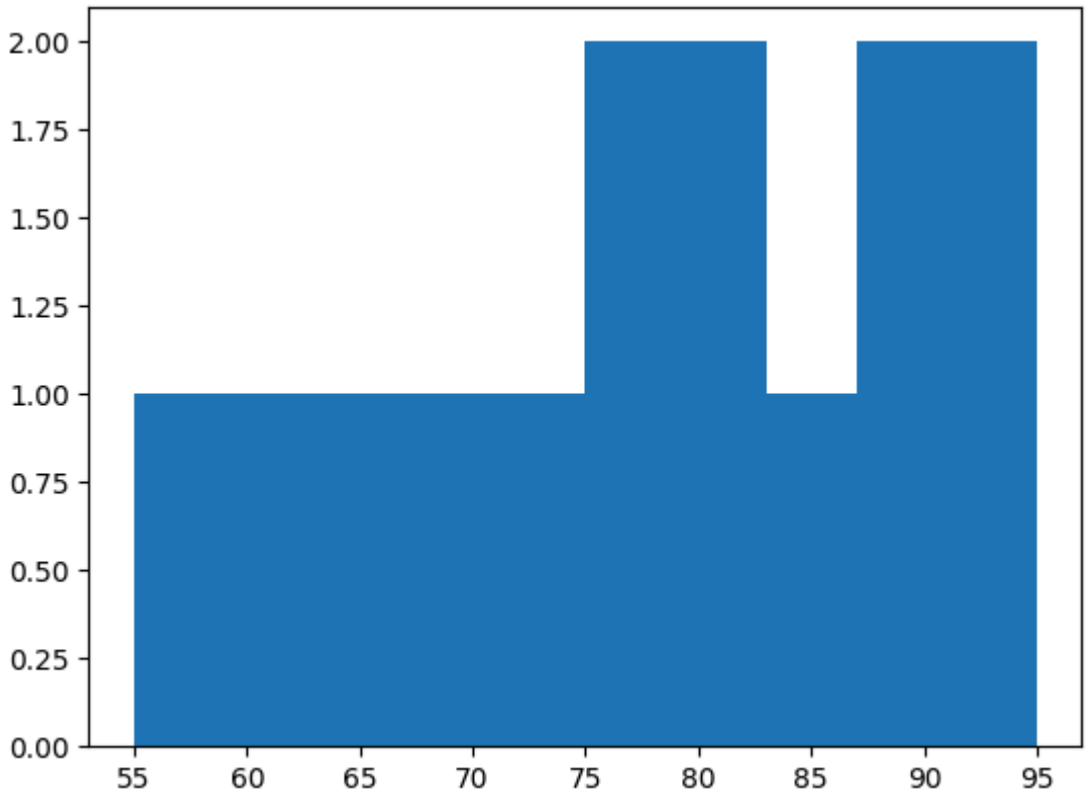
plt.hist(x)

The `hist()` function is used to create a histogram.

Basic Histogram :-

Description: Create a simple histogram.

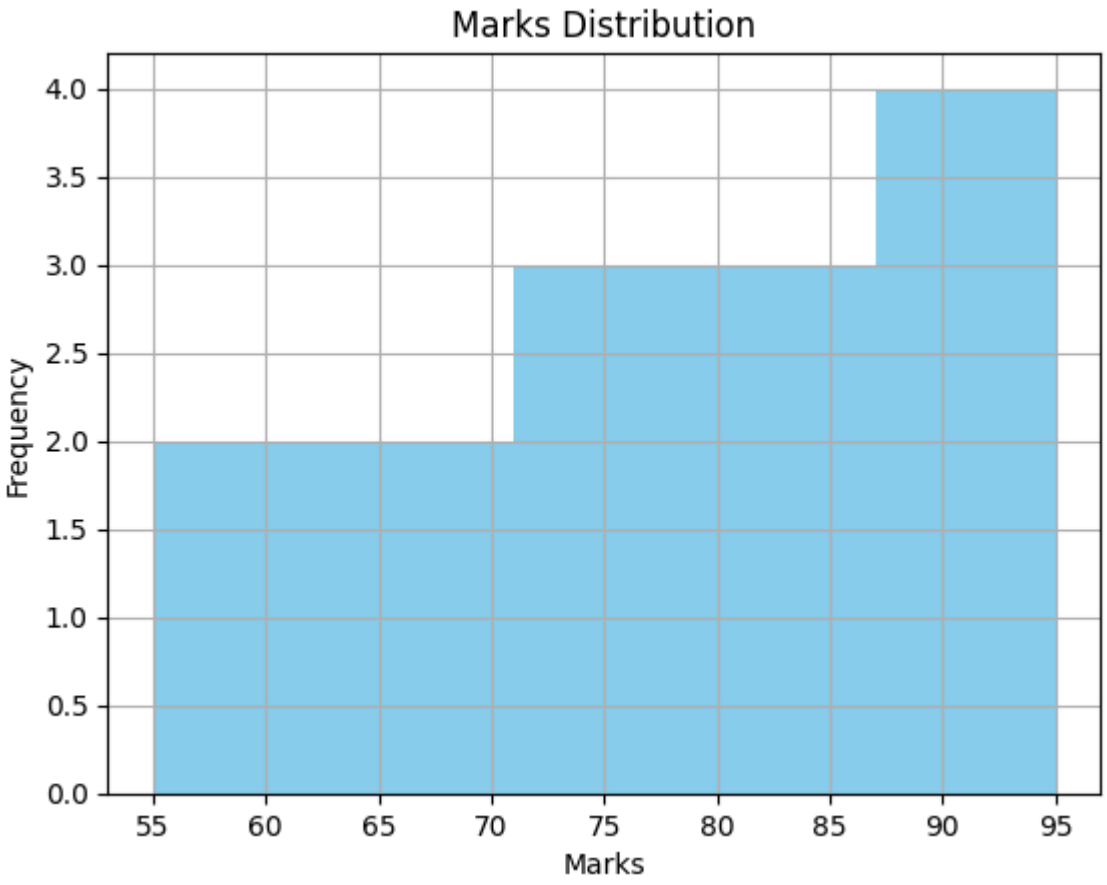
```
import matplotlib.pyplot as plt

marks = [55, 60, 65, 70, 72, 75, 78, 80, 82, 85, 88, 90, 92, 95]

plt.hist(marks)

plt.show()
```



Complete Histogram :-

Description: Add a title, axis labels, and a grid.

```
plt.hist(marks, bins=5, color="skyblue")

plt.title("Marks Distribution")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.grid()

plt.show()
```



5.Scatter Plot :-

A scatter plot is used to show the relationship between two numerical variables. Each point represents one observation in the dataset.

Syntax :-

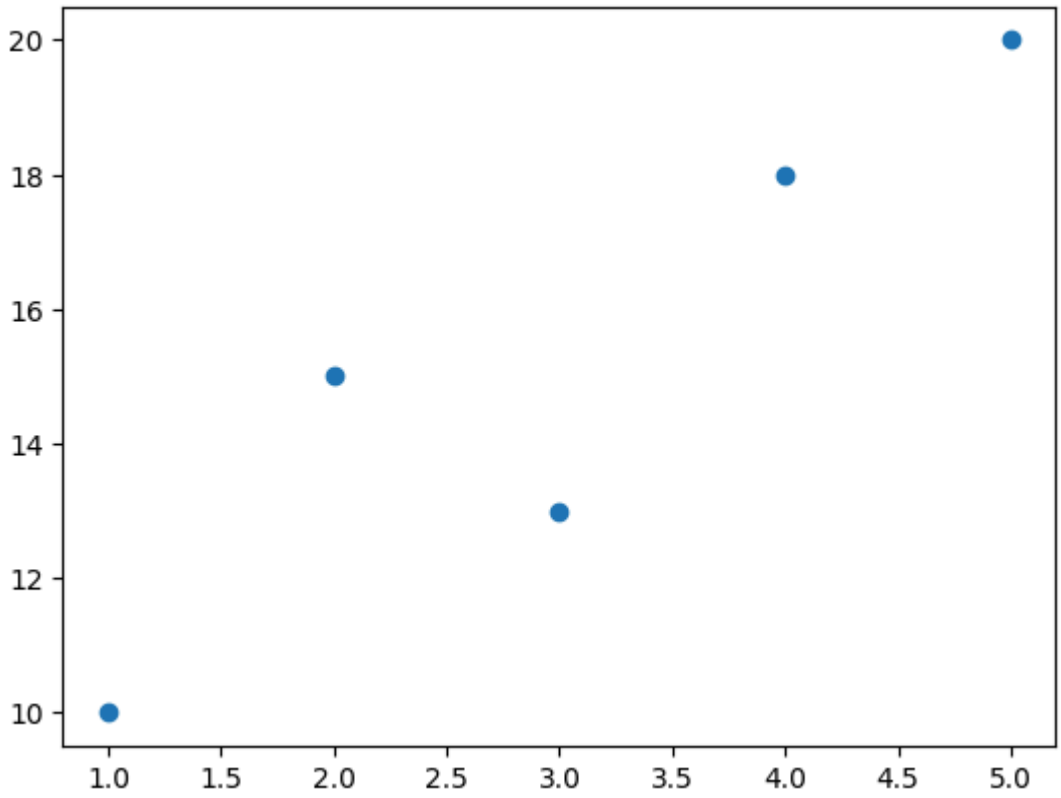
plt.scatter(x, y)

The `scatter()` function is used to create a scatter plot.

Basic Scatter Plot :-

Description: Create a simple scatter plot.

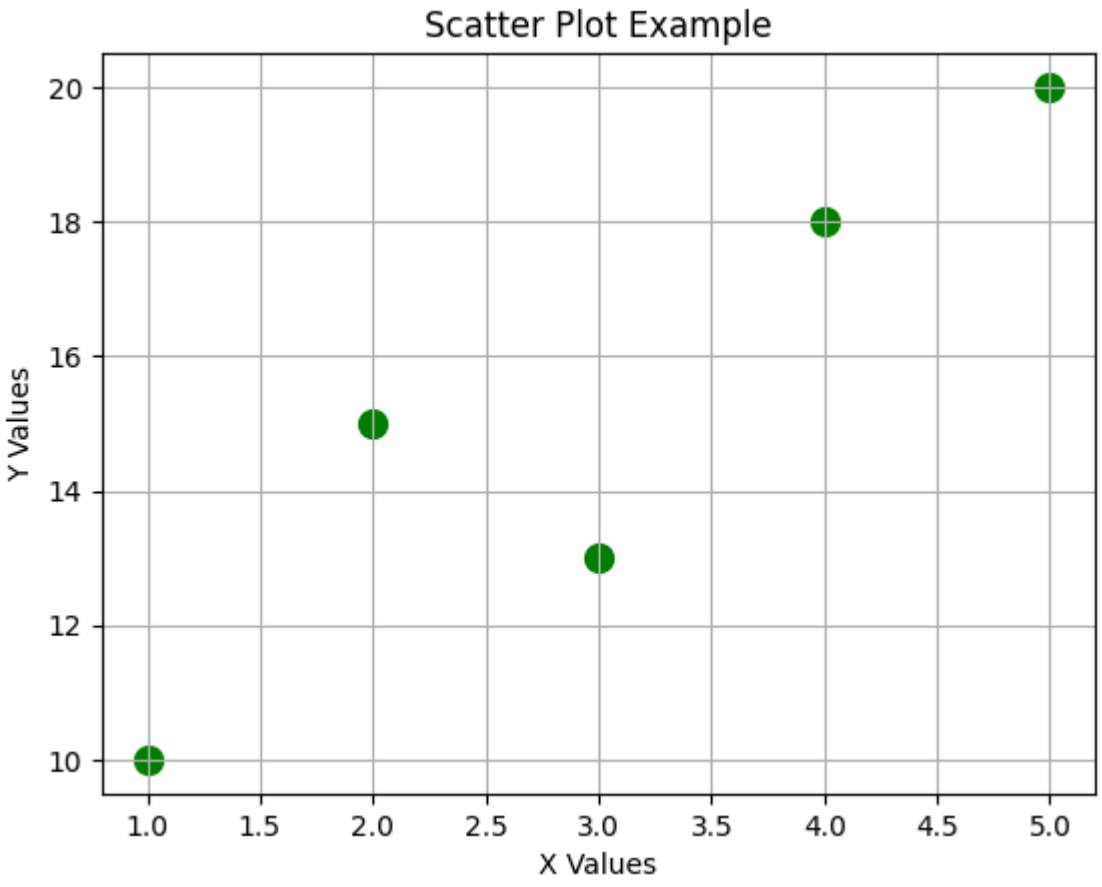
```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [10, 15, 13, 18, 20]

plt.scatter(x, y)

plt.show()
```



Complete Scatter Plot :-

Description: Add a title, axis labels, and a grid.

```
plt.scatter(x, y, color="green", s=100)

plt.title("Scatter Plot Example")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.grid()

plt.show()
```



6.Subplots :-

Subplots allow you to display multiple plots in a single figure. This is useful when comparing different datasets side by side.

Syntax :-

plt.subplot(rows, columns, index)

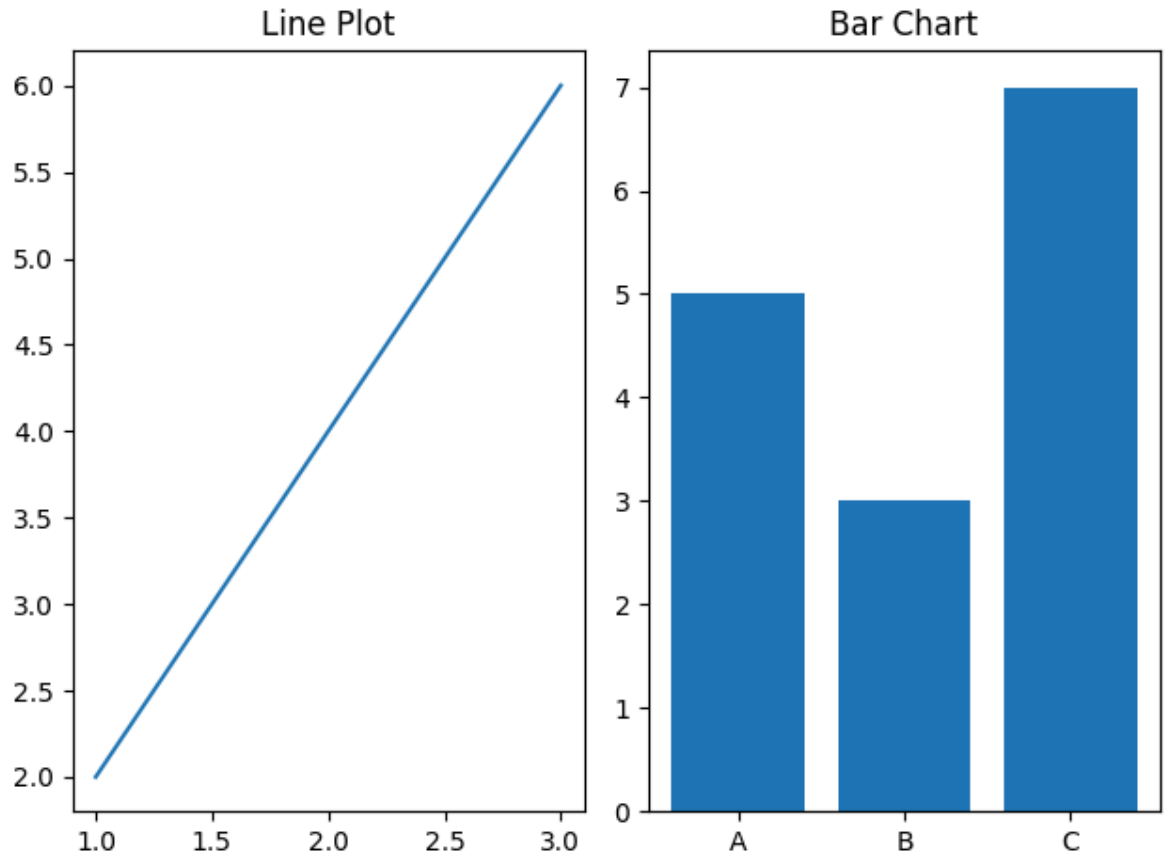
The `subplot()` function creates multiple plots in a grid.

Two Subplots :-

Description: Display a line plot and a bar chart in the same figure.

```
import matplotlib.pyplot as plt

# Line Plot
plt.subplot(1, 2, 1)
plt.plot([1, 2, 3], [2, 4, 6])
plt.title("Line Plot")

# Bar Chart
plt.subplot(1, 2, 2)
```

```
plt.bar(["A", "B", "C"], [5, 3, 7])
plt.title("Bar Chart")

plt.tight_layout()
plt.show()
```

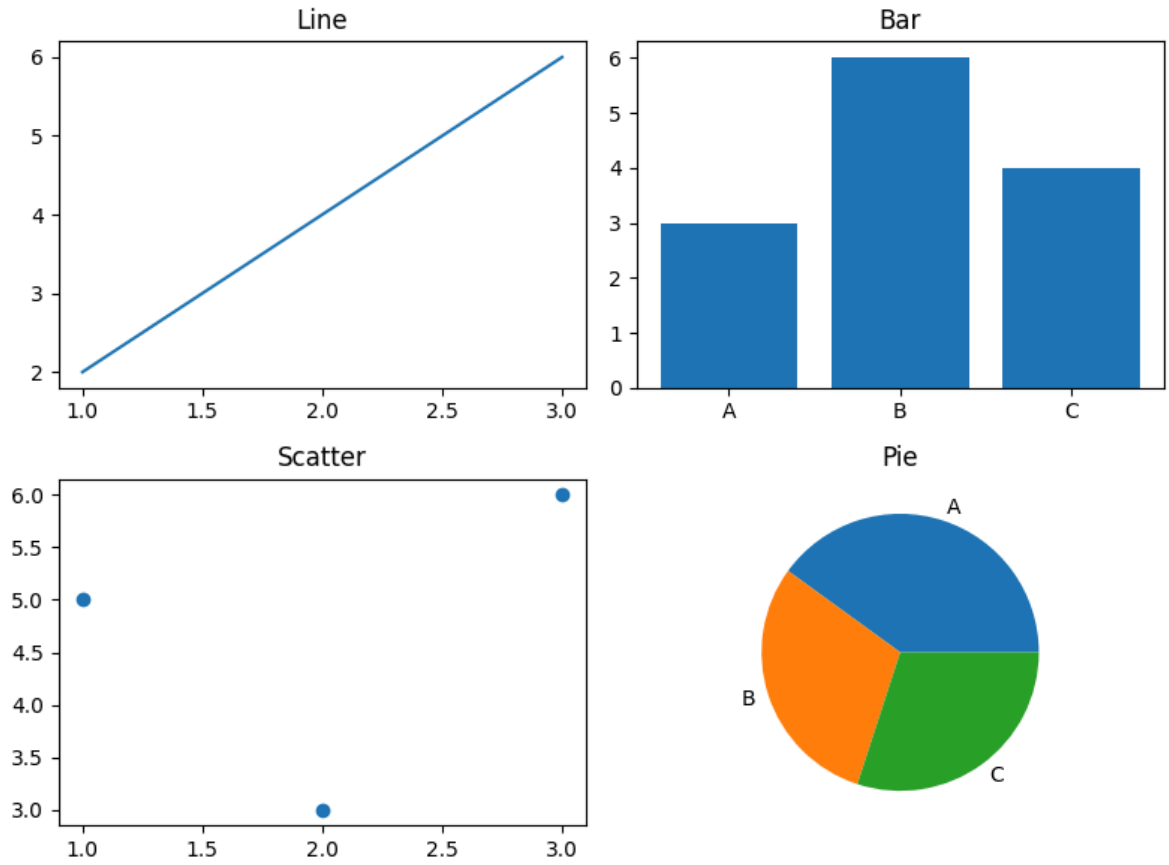


Four Subplots :-

Description: Display four different plots in one figure.

```
plt.figure(figsize=(8, 6))

plt.subplot(2, 2, 1)
plt.plot([1, 2, 3], [2, 4, 6])
plt.title("Line")

plt.subplot(2, 2, 2)
plt.bar(["A", "B", "C"], [3, 6, 4])
plt.title("Bar")

plt.subplot(2, 2, 3)
plt.scatter([1, 2, 3], [5, 3, 6])
plt.title("Scatter")

plt.subplot(2, 2, 4)
plt.pie([40, 30, 30], labels=["A", "B", "C"])
plt.title("Pie")
```

```
plt.tight_layout()
plt.show()
```



Save Figure :-

Matplotlib allows you to save charts as image files. The `savefig()` function saves the current figure in formats such as PNG, JPG, PDF, and SVG.

Syntax :-

plt.savefig("filename.png") - PNG

plt.savefig("filename.pdf") - PDF

Save as PNG :-

Description: Save the graph as a PNG image.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 15, 25]

plt.plot(x, y)
```